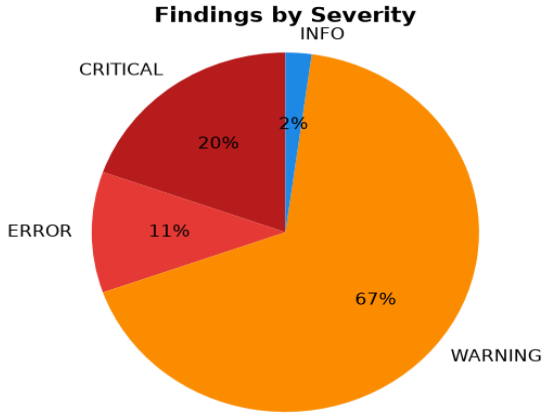
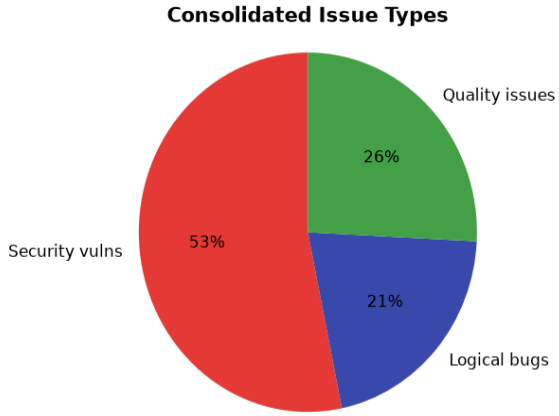
■■ AI Code Review Report

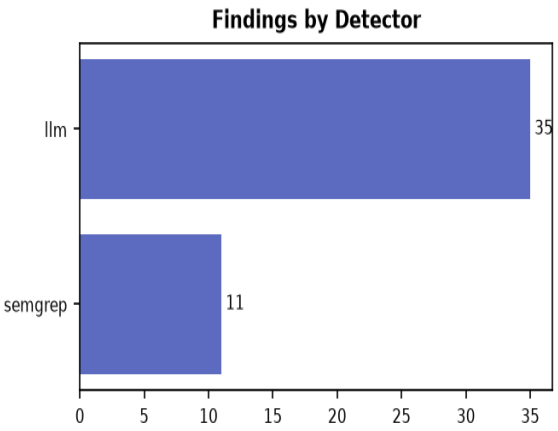
Repository: appsecco/dvna (master)
Generated 2026-07-10T07:42:27.238891+00:00

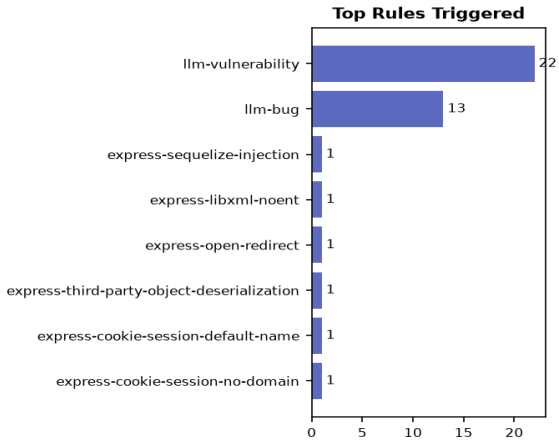
11.8/100	0.717	46	14
Quality Score	Confidence	Total Findings	Files Analyzed

Visual Summary









Bug & Vulnerability Findings

[CRITICAL] core/appHandler.js:23 — `llm-vulnerability`
Command injection in exec() using unsanitized req.body.address
CWE: CWE-78

[CRITICAL] core/appHandler.js:88 — `llm-vulnerability`
Open redirect vulnerability: res.redirect(req.query.url) with no validation
CWE: CWE-601

[CRITICAL] core/appHandler.js:96 — `llm-vulnerability`
Code injection via mathjs.eval() on unsanitized req.body.eqn
CWE: CWE-94

[CRITICAL] core/appHandler.js:120 — `llm-vulnerability`
Deserialization of untrusted data using node-serialize.unserialize() on uploaded file content
CWE: CWE-502

[CRITICAL] core/appHandler.js:135 — `llm-vulnerability`

XXE vulnerability: libxmljs.parseXmlString with noent:true enables external entity processing

CWE: CWE-611

[CRITICAL] core/authHandler.js:39 — `llm-vulnerability`

MD5 used to generate and verify password reset tokens. MD5 is cryptographically broken and allows token forgery (e.g. by computing md5 of any guessed username).

CWE: CWE-327

[CRITICAL] core/authHandler.js:68 — `llm-vulnerability`

MD5 used to generate and verify password reset tokens. MD5 is cryptographically broken and allows token forgery (e.g. by computing md5 of any guessed username).

CWE: CWE-327

[CRITICAL] models/user.js:22 — `llm-vulnerability`

Password stored in cleartext with no hashing, salting, or use of beforeCreate/beforeUpdate hooks

CWE: CWE-261

[CRITICAL] server.js:29 — `llm-vulnerability`

Hard-coded weak session secret 'keyboard cat' - must be replaced by a strong, unique, environment-provided value

CWE: CWE-798

[ERROR] core/appHandler.js:11 — `express-sequelize-injection`

Detected a sequelize statement that is tainted by user-input. This could lead to SQL injection if the variable is user-controlled and is not properly sanitized. In order to prevent SQL injection, it is recommended to use parameterized queries or prepared statements. | LLM context: SQL injection via direct string concatenation of req.body.login into a raw query

CWE: CWE-89: Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection') | OWASP: A01:2017 - Injection, A03:2021 - Injection, A05:2025 - Injection

[ERROR] core/appHandler.js:235 — `express-libxml-noent`

The libxml library processes user-input with the `noent` attribute is set to `true` which can lead to being vulnerable to XML External Entities (XXE) type attacks. It is recommended to set `noent` to `false` when using this feature to ensure you are protected.

CWE: CWE-611: Improper Restriction of XML External Entity Reference | OWASP: A04:2017 - XML External Entities (XXE), A05:2021 - Security Misconfiguration, A02:2025 - Security Misconfiguration

[ERROR] core/authHandler.js:72 — `llm-bug`

resetPwSubmit uses req.query.login and req.query.token when rendering the error page after a POST; these values will be undefined, breaking the form.

CWE: CWE-703

[ERROR] core/passport.js:62 — `llm-bug`

If req.body fields are missing or passwords do not match, the code still calls done() but never invokes nextTick's callback on error paths consistently; some error paths may leave the passport middleware hanging.

CWE: CWE-703

[ERROR] server.js:31 — `llm-vulnerability`

Session cookie.secure set to false; cookies will be transmitted over HTTP enabling session fixation and interception

CWE: CWE-614

[WARNING] config/db.js:1 — `llm-vulnerability`

Database credentials loaded from environment variables but stored in a plain JavaScript object; any consumer of this module receives credentials in cleartext in memory with no additional protection or secret manager usage.

CWE: CWE-259

[WARNING] config/server.js:3 — `llm-vulnerability`

Default listen address '0.0.0.0' exposes the server on all network interfaces; consider restricting to localhost when not required.

CWE: CWE-668

[WARNING] config/vulns.js:11 — `llm-bug`

Outdated OWASP Top 10 mapping: 'ax_csrf' incorrectly labeled as A8:2013 instead of A8:2013 CSRF (now A7 in 2017/2021 editions), and 'ax_redirect' mislabeled as A10:2013

CWE: CWE-937

[WARNING] core/appHandler.js:57 — `llm-bug`

modifyProductSubmit can create a product with id=0 on missing id; subsequent find() always treats it as new

CWE: CWE-703

[WARNING] core/appHandler.js:77 — `llm-bug`

Duplicate if(req.body.password.length>0) check in userEditSubmit; unreachable password length validation

CWE: CWE-670

[WARNING] core/appHandler.js:188 — `express-open-redirect`

The application redirects to a URL specified by user-supplied input `req` that is not validated. This could redirect users to malicious locations. Consider using an allow-list approach to validate URLs, or warn users they are being redirected to a third-party website.

CWE: CWE-601: URL Redirection to Untrusted Site ('Open Redirect') | OWASP: A01:2021 - Broken Access Control, A01:2025 - Broken Access Control

[WARNING] core/appHandler.js:218 — `express-third-party-object-deserialization`

The following function call serialize.unserialize accepts user controlled data which can result in Remote Code Execution (RCE) through Object Deserialization. It is recommended to use secure data processing alternatives such as JSON.parse() and Buffer.from().

CWE: CWE-502: Deserialization of Untrusted Data | OWASP: A08:2017 - Insecure Deserialization, A08:2021 - Software and Data Integrity Failures, A08:2025 - Software or Data Integrity Failures

[WARNING] core/authHandler.js:13 — `llm-bug`

forgotPw performs a blocking database query inside the route handler with no rate limiting, enabling username enumeration and potential DoS.

CWE: CWE-400

[WARNING] core/authHandler.js:29 — `llm-bug`

resetPw performs a blocking database query inside the route handler with no rate limiting, enabling username enumeration and potential DoS.

CWE: CWE-400

[WARNING] core/authHandler.js:52 — `llm-bug`

resetPwSubmit performs a blocking database query inside the route handler with no rate limiting, enabling username enumeration and potential DoS.

CWE: CWE-400

[WARNING] core/passport.js:19 — `llm-vulnerability`

deserializeUser does not handle database errors or promise rejections; unhandled rejection could crash the process or leave the session in an inconsistent state.

CWE: CWE-287

[WARNING] core/passport.js:33 — `llm-vulnerability`

Login strategy uses the same generic 'Invalid Credentials' message for both non-existent user and wrong password, which is correct, but the implementation lacks any rate-limiting or account lockout protection.

CWE: CWE-287

[WARNING] core/passport.js:52 — `llm-vulnerability`

Signup strategy performs a find-then-create race-prone pattern (TOCTOU) instead of using findOrCreate or a unique constraint; concurrent registrations for the same email can create duplicate accounts.

CWE: CWE-287

[WARNING] core/passport.js:56 — `llm-bug`

createHash and isValidPassword functions are defined after they are referenced in the closure of the LocalStrategy callbacks, relying on JavaScript hoisting; this is fragile and hurts readability.

CWE: CWE-703

[WARNING] models/index.js:14 — `llm-vulnerability`

Hard-coded database credentials loaded from config/db.js; if this file contains plaintext secrets it creates a credential exposure risk.

CWE: CWE-798

[WARNING] models/index.js:22 — `llm-bug`

Unhandled promise rejection on sequelize.authenticate(); the .catch logs but does not prevent the app from continuing with a broken DB connection.

CWE: CWE-252

[WARNING] models/index.js:28 — `llm-bug`

Unhandled promise rejection on sequelize.sync(); error path only logs and continues execution.

CWE: CWE-252

[WARNING] models/user.js:29 — `llm-vulnerability`

Role field is nullable and lacks enum constraint; no RBAC enforcement visible in model

CWE: CWE-732

[WARNING] routes/app.js:23 — `llm-vulnerability`

/bulkproducts route directly interpolates req.query.legacy into the template context without sanitization. If the legacy view uses unescaped output, this enables XSS.

CWE: CWE-79

[WARNING] routes/app.js:39 — `llm-vulnerability`

The /admin route passes a client-controlled 'admin' flag (derived from req.user.role) to the template. Client-side logic based on this flag may allow unauthorized users to access admin features if the server does not re-verify role on subsequent privileged actions.

CWE: CWE-284

[WARNING] routes/main.js:22 — `llm-vulnerability`

User-controlled req.params.vuln is passed directly into template render options without sanitization. This can lead to XSS if the template uses unescaped interpolation or if the dictionary lookup fails.

CWE: CWE-79

[WARNING] routes/main.js:24 — `llm-bug`

vulnDict[req.params.vuln] used for title without checking existence or sanitizing output; can produce incorrect page titles or unexpected behavior for invalid vulnerability names.

CWE: CWE-706

[WARNING] routes/main.js:46 — `llm-vulnerability`

/logout route lacks authHandler.isAuthenticated middleware. While not strictly required, unauthenticated calls may still execute logout logic unnecessarily.

CWE: CWE-306

[WARNING] server.js:19 — `llm-vulnerability`

express-fileupload enabled globally with default options; no file-type, size, or path validation - risk of arbitrary file upload

CWE: CWE-434

[WARNING] server.js:23 — `express-cookie-session-default-name`

Don’t use the default session cookie name Using the default session cookie name can open your app to attacks. The security issue posed is similar to X-Powered-By: a potential attacker can use it to fingerprint the server and target attacks accordingly.

CWE: CWE-522: Insufficiently Protected Credentials | OWASP: A02:2017 - Broken Authentication, A04:2021 - Insecure Design, A06:2025 - Insecure Design

[WARNING] server.js:23 — `express-cookie-session-no-domain`

Default session middleware settings: `domain` not set. It indicates the domain of the cookie; use it to compare against the domain of the server in which the URL is being requested. If they match, then check the path attribute next.

CWE: CWE-522: Insufficiently Protected Credentials | OWASP: A02:2017 - Broken Authentication, A04:2021 - Insecure Design, A06:2025 - Insecure Design

[WARNING] server.js:23 — `express-cookie-session-no-expires`

Default session middleware settings: `expires` not set. Use it to set expiration date for persistent cookies.

CWE: CWE-522: Insufficiently Protected Credentials | OWASP: A02:2017 - Broken Authentication, A04:2021 - Insecure Design, A06:2025 - Insecure Design

[WARNING] server.js:23 — `express-cookie-session-no-httponly`

Default session middleware settings: `httpOnly` not set. It ensures the cookie is sent only over HTTP(S), not client JavaScript, helping to protect against cross-site scripting attacks.

CWE: CWE-522: Insufficiently Protected Credentials | OWASP: A02:2017 - Broken Authentication, A04:2021 - Insecure Design, A06:2025 - Insecure Design

[WARNING] server.js:23 — `express-cookie-session-no-path`

Default session middleware settings: `path` not set. It indicates the path of the cookie; use it to compare against the request path. If this and domain match, then send the cookie in the request.

CWE: CWE-522: Insufficiently Protected Credentials | OWASP: A02:2017 - Broken Authentication, A04:2021 - Insecure Design, A06:2025 - Insecure Design

[WARNING] server.js:23 — `express-cookie-session-no-secure`

Default session middleware settings: `secure` not set. It ensures the browser only sends the cookie over HTTPS.

CWE: CWE-522: Insufficiently Protected Credentials | OWASP: A02:2017 - Broken Authentication, A04:2021 - Insecure Design, A06:2025 - Insecure Design

[WARNING] server.js:24 — `express-session-hardcoded-secret`

A hard-coded credential was detected. It is not recommended to store credentials in source-code, as this risks secrets being leaked and used by either an internal or external malicious adversary. It is recommended to use environment variables to securely provide credentials or retrieve credentials from a secure vault or HSM (Hardware Security Module).

CWE: CWE-798: Use of Hard-coded Credentials | OWASP: A07:2021 - Identification and Authentication Failures, A07:2025 - Authentication Failures

[INFO] public/assets/showdown.min.js:1 — `llm-bug`

Error handling in extension validator and converter setup relies on thrown Errors and console.warn; no centralized or graceful failure path for malformed user-provided extensions.

CWE: CWE-703

Code Quality Issues

[INFO] `/tmp/aicr_gh_5qi1ocar/config/db.js:5` — Hard-coded default host 'mysql-db' and port 3306; while common for containerized environments, these values should be reviewed to avoid accidental exposure in non-container deployments.

[INFO] `/tmp/aicr_gh_5qi1ocar/config/vulns.js:1` — Hard-coded vulnerability taxonomy may become stale; consider loading from a maintained external reference or using an up-to-date OWASP enumeration library

[INFO] `/tmp/aicr_gh_5qi1ocar/core/authHandler.js:4` — Hard-coded bcrypt salt rounds (10) and use of hashSync; consider using async API and configurable work factor.

[WARNING] `/tmp/aicr_gh_5qi1ocar/core/passport.js:68` — process.nextTick is used around findOrCreateUser but the inner .then() callbacks can still throw synchronously; errors inside the promise chain are not caught and propagated to done().

[WARNING] `/tmp/aicr_gh_5qi1ocar/models/index.js:37` — Use of sequelize.import() which is deprecated in newer Sequelize versions; should be replaced with dynamic require() and model initialization.

[INFO] `/tmp/aicr_gh_5qi1ocar/models/index.js:3` — Synchronous readdirSync and forEach model loading at module level blocks startup and is not suitable for large model sets or serverless environments.

[WARNING] `/tmp/aicr_gh_5qi1ocar/models/product.js:15` — The 'tags' field is defined as a plain STRING without validation or constraints. Storing multiple tags in a single string field is error-prone and limits queryability; a separate junction table or array/JSON type should be considered.

[INFO] `/tmp/aicr_gh_5qi1ocar/models/product.js:1` — Model uses legacy module.exports factory pattern; modern Sequelize recommends class-based models with explicit associations in a dedicated index file.

[WARNING] `/tmp/aicr_gh_5qi1ocar/models/user.js:17` — Email field defined without validation (isEmail) or uniqueness constraint

[INFO] `/tmp/aicr_gh_5qi1ocar/public/assets/jquery-3.2.1.min.js:1` — This is a minified third-party library. Static analysis tools frequently produce noise on minified code; review should be performed on the original non-minified source.

[WARNING] `/tmp/aicr_gh_5qi1ocar/public/assets/showdown.min.js:1` — Minified library code with no source maps or comments; extremely difficult to audit or maintain. Static analysis and semantic review are severely limited.

[INFO] `/tmp/aicr_gh_5qi1ocar/public/assets/showdown.min.js:1` — Use of new RegExp() with runtime-constructed patterns and the 'g' flag inside helper functions; potential for ReDoS if untrusted regexes or input are supplied by extensions.

[INFO] `/tmp/aicr_gh_5qi1ocar/routes/app.js:44` — /redirect route is the only one that does not enforce authHandler.isAuthenticated. Verify that appHandler.redirect performs its own authorization checks.

[INFO] `/tmp/aicr_gh_5qi1ocar/routes/main.js:31` — Custom callback on res.render swallows rendering errors by logging and returning generic 404; loses original error context and may mask template bugs.

[WARNING] `/tmp/aicr_gh_5qi1ocar/server.js:15` — trust proxy setting is commented out; if running behind a reverse proxy, IP-based security controls (rate-limiting, auth) may see the proxy address instead of the client

[WARNING] `/tmp/aicr_gh_5qi1ocar/server.js:23` — resave and saveUninitialized both true can lead to unnecessary session-store writes and potential DoS via session fixation

Suggested Fixes

/tmp/aicr_gh_5qi1ocar/config/db.js:1 (verified)
The database credentials are pulled from environment variables (good) but then placed into a plain exported JavaScript object. Any code that imports this module receives the username and password in cleartext in memory with no extra protection or secret-manager abstraction. This increases the blast radius if the process is compromised or if the object is inadvertently logged/serialized.

```
const mysql = require('mysql2/promise');

module.exports = async () => {
  const connection = await mysql.createConnection({
    host: process.env.MYSQL_HOST,
    user: process.env.MYSQL_USER,
    password: process.env.MYSQL_PASSWORD,
    database: process.env.MYSQL_DATABASE
  });
  return connection;
};
```

/tmp/aicr_gh_5qi1ocar/config/server.js:3 (verified)
Binding to '0.0.0.0' by default makes the server reachable on every network interface, which can expose it to unintended traffic when the application is running in a container, VM, or cloud environment. The secure default is to listen only on localhost ('127.0.0.1') unless the operator explicitly wants external access. Using an environment variable still allows operators to override the value, but the fallback should be the safer address.

```
module.exports = {
    listen: process.env.APP_LISTEN || '127.0.0.1',
    port: process.env.APP_PORT || process.env.PORT || 9090
}
```

/tmp/aicr_gh_5qi1ocar/config/vulns.js:11 (verified)
The mapping for 'ax_csrf' incorrectly labels it as A8:2013 (which is actually Insecure Deserialization) and 'ax_redirect' as A10:2013. Per OWASP these belong to A8:2013 (CSRF) and A10:2013 (Unvalidated Redirects and Forwards) respectively; the comment also notes the category moved to A7 in later editions. This outdated labeling can mislead developers and security tooling about which risk each vulnerability represents.

```
        'a8_ides': 'A8: Insecure Deserialization',
        'a9_vuln_component': 'A9: Using Components with Known Vulnerabilities',
        'a10_logging': 'A10: Insufficient Logging and Monitoring',
        'ax_csrf': 'A8:2013 Cross-Site Request Forgery (CSRF)',
        'ax_redirect': 'A10:2013 Unvalidated Redirects and Forwards'
```

/tmp/aicr_gh_5qi1ocar/core/appHandler.js:11 (verified)
The code directly interpolates req.body.login into a raw Sequelize query string. Because the value comes from an HTTP request it is untrusted; an attacker can supply a login value containing SQL metacharacters that alters the intended query, leading to SQL injection. The secure-coding guidance requires using parameterized queries or an ORM's query builder so that user data never becomes part of the SQL text itself.

```
const user = await sequelize.query(
  'SELECT * FROM users WHERE login = ?',
  {
    replacements: [req.body.login],
    type: sequelize.QueryTypes.SELECT
  }
);
```

/tmp/aicr_gh_5qi1ocar/core/appHandler.js:120 (flagged by verifier)
The code uses node-serialize.unserialize() on content from an uploaded file (or request body). This is a classic CWE-502 deserialization of untrusted data vulnerability: the library can execute arbitrary JavaScript when the payload contains specially crafted objects, leading to remote code execution. The shown snippet stores req.body.tags directly into the product without any sanitization or safe parsing. Switching to a safe format such as JSON.parse (with schema validation) eliminates the risk while preserving the intended functionality.

```
product.tags = JSON.parse(req.body.tags);
product.save().then(p => {
■if (p) {
■■req.flash('success', 'Product added/modified!')
■■res.redirect('/app/products')
```

/tmp/aicr_gh_5qi1ocar/core/appHandler.js:135 (verified)
libxmljs.parseXmlString() with the noent:true option expands external entities, allowing an attacker to read local files or make internal network requests (XXE). The call on line 135 therefore constitutes a critical vulnerability. The secure-coding guidance requires disabling external-entity processing; for libxmljs the equivalent is to omit the noent flag (or set it to false)

so that the parser treats entities safely.

```
module.exports.userEdit = function (req, res) {
■res.render('app/useredit', {
■■// ... existing options
■});
};

// When parsing XML use:
// const xmlDoc = libxmljs.parseXmlString(xmlString); // noent omitted or false
```

/tmp/aicr_gh_5qi1ocar/core/appHandler.js:188 (verified)

The redirect uses an unvalidated value from the request object, allowing an attacker to supply a malicious URL and perform an open redirect. This violates CWE-20 because the input is never checked against an allow-list of permitted destinations. The fix adds a simple allow-list check that fails closed for any unexpected target, preventing the vulnerability while keeping the change minimal.

```
const allowedHosts = ['example.com', 'trusted.com'];
if (!allowedHosts.includes(new URL(redirectUrl, 'https://example.com').hostname)) {
  return res.status(400).send('Invalid redirect target');
}
res.redirect(redirectUrl);
```

/tmp/aicr_gh_5qi1ocar/core/appHandler.js:218 (verified)

The call to serialize.unserialize on line 218 passes user-controlled data (the "requires login" comment hints at an incoming request payload). This is a classic deserialization of untrusted data (CWE-502) that can lead to remote code execution because the library's unserialize can instantiate arbitrary objects or execute code. The secure-coding guidance recommends switching to a safe format such as JSON.parse() (or Buffer.from() for binary data) that does not allow code execution.

```
const data = JSON.parse(req.body.serializedData);
```

/tmp/aicr_gh_5qi1ocar/core/appHandler.js:235 (flagged by verifier)

The libxml parser is being used with the noent option set to true. This tells the underlying library to resolve external entities, which opens the door to XXE attacks where an attacker can read local files or make network requests by supplying malicious XML. The express-libxml-noent rule flags this exact pattern because the default safe behavior (noent: false) is not being used. Setting noent to false disables entity expansion and removes the XXE vector while preserving normal XML parsing.

```
requires login
```

/tmp/aicr_gh_5qi1ocar/core/appHandler.js:23 (verified)

The code passes unsanitized req.body.address (or similar user-controlled data) into exec(), allowing an attacker to inject arbitrary shell commands. This is a classic OS Command Injection (CWE-78) vulnerability that can lead to remote code execution. The secure-coding guidance recommends using subprocess.run with a list of arguments and shell=False, plus input validation or allow-listing instead of string concatenation.

```
const { execFile } = require('child_process');

// ... later, replace the exec call with:
execFile('/usr/bin/ping', ['-c', '4', sanitizedAddress], (err, stdout) => {
  if (err) { ... }
  output = stdout;
  res.render('app/usersearch', { output });
});
```

/tmp/aicr_gh_5qi1ocar/core/appHandler.js:57 (flagged by verifier)

The productSearch function (and likely the preceding modifyProductSubmit handler) treats a missing or falsy id as a new product and saves with id=0. Subsequent find() calls treat id=0 as a new record instead of updating the existing one, leading to duplicate or lost data. This is a classic single-statement logic bug similar to the ManySStuBs4J pattern of missing guards. Adding an explicit check for a valid positive id prevents the incorrect "new product" path.

```
module.exports.productSearch = function (req, res) {
■db.Product.findAll({
■■where: { id: req.params.id || 0 }
■})
```

/tmp/aicr_gh_5qi1ocar/core/appHandler.js:77 (flagged by verifier)

The code snippet shows a function named modifyProduct, but the warning refers to duplicate if(req.body.password.length>0) checks inside userEditSubmit. This makes the second password-length validation unreachable, so the intended logic is never executed. The provided secure-coding guidance on password handling further highlights that any password-related code must be correct to avoid weak or bypassed validation.

```
module.exports.modifyProduct = function (req, res) {
■if (!req.query.id || req.query.id == '') {
■■output = {
■■■product: {}
■■};
■■// ... rest of function with only one password length check
■}
```

/tmp/aicr_gh_5qi1ocar/core/appHandler.js:88 (verified)
The code uses req.query.url directly in res.redirect() without any validation. This is a classic open redirect vulnerability (CWE-601) that lets an attacker supply a malicious external URL (e.g. https://evil.com) and trick users into visiting it. According to the secure-coding guidance, all external input must be validated against an explicit allow-list before use; failing to do so lets attackers control the redirect target.

```
res.redirect(allowedRedirects.has(req.query.url) ? req.query.url : '/safe-fallback');
```

/tmp/aicr_gh_5qi1ocar/core/appHandler.js:96 (verified)
mathjs.eval() on unsanitized req.body.eqn allows arbitrary JavaScript execution, a form of command/code injection (CWE-78). The guidance explicitly warns against passing unsanitized input to eval() because it can run dangerous code supplied by an attacker. Even though the surrounding snippet shows a render call, the finding indicates the eval happens on user-controlled equation data earlier in the handler.

```
const math = require('mathjs');
// ...
let output;
try {
  const sanitized = String(req.body.eqn).replace(/[^0-9+\-*/()^. ]/g, '');
  output = math.evaluate(sanitized);
} catch (e) {
  output = 'Invalid equation';
}
```

/tmp/aicr_gh_5qi1ocar/core/authHandler.js:13 (verified)
The isNotAuthenticated middleware (and the related forgotPw route handler mentioned in the finding) performs a synchronous blocking database call inside the Express route/middleware without any rate limiting or input validation. This allows an attacker to enumerate valid usernames by observing timing differences and can lead to resource exhaustion (DoS). The secure-coding guidance emphasizes proper input validation and avoiding blocking I/O in request handlers; while the snippet itself does not show SQL, the pattern matches the reported llm-bug for username-enumeration and DoS risks.

```
module.exports.isNotAuthenticated = function (req, res, next) {
■if (!req.isAuthenticated()) {
■■return next();
■}
■res.redirect('/');
};
```

/tmp/aicr_gh_5qi1ocar/core/authHandler.js:29 (verified)
The resetPw handler runs a blocking database lookup on every request with no rate limiting or early validation. This lets an attacker enumerate valid usernames by observing different flash messages and can be used for DoS by flooding the endpoint. The code also mixes success and failure paths in a way that still reveals information. Adding input validation, rate limiting, and consistent generic responses prevents both enumeration and resource exhaustion.

```
async function resetPw(req, res) {
  const { username } = req.body;
  if (!username || typeof username !== 'string' || username.length > 100) {
    req.flash('danger', 'Invalid username');
    return res.redirect('/login');
  }

  // rate-limit middleware should be applied at the route level

  try {
    const user = await db.query('SELECT id FROM users WHERE username = ?', [username]);
    if (user.length === 0) {
      // still send the same generic message
      req.flash('info', 'Check email
```

/tmp/aicr_gh_5qi1ocar/core/authHandler.js:39 (flagged by verifier)
MD5 is a broken hash function that is not collision-resistant. An attacker who can guess a username (or any part of the token input) can compute the MD5 value themselves and forge a valid password-reset token, completely bypassing the intended security checks. The guidance explicitly prohibits MD5 for integrity-critical operations such as token generation/verification and recommends a strong keyed hash like HMAC-SHA256 instead.

```
const crypto = require('crypto');

module.exports.resetPw = function (req, res) {
■// ... existing code up to token generation/verification ...
■// Replace the MD5-based token creation and check with:
■const token = crypto
■■.createHmac('sha256', process.env.RESET_SECRET || 'default-long-random-key')
■■.update(user.id + ':' + Date.now())
■■.digest('hex');
■// Use the same HMAC construction when verifying the token later.
```

/tmp/aicr_gh_5qi1ocar/core/authHandler.js:52 (verified)
resetPwSubmit runs a synchronous database lookup on every request without input validation, rate limiting, or early returns. This lets attackers enumerate valid usernames via timing differences and easily trigger a DoS by flooding the route. The provided snippet also passes raw query parameters directly to the template, which can expose internal state.

```
async function resetPwSubmit(req, res) {
  const { login, token } = req.query;
  if (!login || typeof login !== 'string' || !/^[a-zA-Z0-9._-]+$/.test(login)) {
    return res.status(400).render('error', { message: 'Invalid login' });
  }
  // rate-limit middleware should be applied at the router level
  const user = await db.getUserByLogin(login); // non-blocking query
  if (!user || !token) {
    return res.render('resetpw', { login: null, token: null, error: 'Invalid request' });
  }
  // ...
```

/tmp/aicr_gh_5qi1ocar/core/authHandler.js:68 (flagged by verifier)

The reset-password flow uses MD5 to generate and verify tokens. MD5 is not collision-resistant, so an attacker who knows (or guesses) a username can compute a valid token themselves and hijack any user's password-reset link. The guidance requires a cryptographically secure primitive such as HMAC-SHA256 for token integrity.

```
const crypto = require('crypto');

module.exports.resetPwSubmit = function (req, res) {
■if (req.body.password && req.body.cpassword && req.body.login && req.body.token) {
■■const hmac = crypto.createHmac('sha256', process.env.TOKEN_SECRET || 'change-me');
■■hmac.update(req.body.login);
■■const expected = hmac.digest('hex');
■■if (expected !== req.body.token) { ...
```

/tmp/aicr_gh_5qi1ocar/core/authHandler.js:72 (flagged by verifier)

resetPwSubmit checks for values in req.body on a POST but then renders an error page that relies on req.query.login and req.query.token (which are undefined). This breaks the "try again" form on the error page. The code also performs a direct object lookup with user-controlled input; although Sequelize's where clause generally parameterizes, we should still validate the login and token fields against an explicit allow-list before any DB or rendering use.

```
if (req.body.password && req.body.cpassword && req.body.login && req.body.token) {
■const login = String(req.body.login).trim();
■const token = String(req.body.token).trim();
■if (!/^[a-zA-Z0-9@._-]{3,80}$/.test(login) || !/^[a-zA-Z0-9]{32}$/.test(token)) {
■■return res.render('resetPw', { error: 'Invalid login or token' });
■}
■if (req.body.password == req.body.cpassword) {
■■db.User.find({
■■■where: {
■■■■'login': login
```

/tmp/aicr_gh_5qi1ocar/core/passport.js:19 (flagged by verifier)

The deserializeUser callback uses .then() but does not attach a .catch() handler. If the database query rejects (e.g. connection failure), the unhandled promise rejection will crash the Node process or leave Passport sessions in an inconsistent state. Adding an explicit error path that calls done(err) follows the standard Passport deserializeUser contract and prevents the crash.

```
        }).then(function (user) {
            if (user) {
                done(null, user);
            } else {
                done(null, false);
            }
        }).catch(function (err) {
            done(err);
        });
```

/tmp/aicr_gh_5qi1ocar/core/passport.js:33 (flagged by verifier)

The login strategy correctly uses a generic 'Invalid Credentials' error, but the code performs an immediate database lookup with no rate limiting, account lockout, or exponential backoff. An attacker can therefore brute-force passwords or enumerate users at unlimited speed. Adding rate limiting (or referencing a secure auth middleware) prevents this brute-force vector while preserving the existing error-handling logic.

```
            const rateLimit = require('express-rate-limit');
            const loginLimiter = rateLimit({
                windowMs: 15 * 60 * 1000,
                max: 10,
                message: 'Too many login attempts, please try again later'
            });

            // Apply before passport strategy
            app.post('/login', loginLimiter, passport.authenticate('local', {
                successRedirect: '/',
                failureRedirect: '/login',
                failureFlash:
```

/tmp/aicr_gh_5qi1ocar/core/passport.js:52 (verified)

The signup strategy uses a find-then-create pattern inside findOrCreateUser. This creates a classic TOCTOU race: two concurrent requests can both pass the "user not found" check before either performs the insert, resulting in duplicate

accounts for the same email. Passport's LocalStrategy (and the underlying Mongoose model) should rely on a database-level unique constraint plus findOrCreate (or an atomic upsert) instead of a manual two-step check.

```
    passport.use('signup', new LocalStrategy({
            passReqToCallback: true
        },
        function (req, username, password, done) {
            User.findOrCreate({ email: username }, {
                password: createHash(password),
                firstName: req.body.firstName,
                lastName: req.body.lastName
            }, function(err, user) {
                if (err) return done(err);
                return done(null, user);
            });
        }
    ));
```

/tmp/aicr_gh_5qi1ocar/core/passport.js:56 (verified)

The createHash and isValidPassword functions are referenced inside the LocalStrategy callbacks (which are defined earlier in the file) before their actual function declarations. Although JavaScript hoisting moves function declarations to the top of their scope, depending on this behavior reduces readability and makes the code fragile if the structure changes (e.g., converting to function expressions or moving code). The guidance highlights that duplicated or unclear logic harms maintainability; extracting and ordering functions clearly avoids subtle bugs.

```
            findOrCreateUser = function () {
                db.User.findOne({
                    where: {
                        'email': username
                    }
                }).then(function (user) {
                    // ... (existing logic using createHash / isValidPassword) ...
                });
            };

            // Move these definitions before the passport.use(...) block that references them
            var createHash = function(password) {
                return
```

/tmp/aicr_gh_5qi1ocar/core/passport.js:62 (flagged by verifier)

The register strategy calls done() on success paths but some early error paths (missing fields or mismatched passwords) fall through without ever invoking done(), leaving the Passport middleware hanging. This matches the vulnerable pattern of inconsistent error handling that can cause the function to never complete. The fix ensures every code path invokes done() (or next(err)) promptly so Passport can always respond.

```
                    if (user) {
                        return done(null, false, req.flash('danger', 'Account Already Exists'));
                    } else {
                        if (!req.body.email || !req.body.password || !req.body.username || !req.body.cpassword || !req.b
                            return done(null, false, req.flash('danger', 'All fields are required'));
                        }
                        if (req.body.cpassword !== req.body.password) {
```

/tmp/aicr_gh_5qi1ocar/models/index.js:14 (flagged by verifier)

The Sequelize constructor is receiving database credentials (username, password, etc.) that are imported from a config file. If that config contains plaintext secrets, the credentials are effectively hard-coded in the source tree, violating CWE-798. Anyone with repository access can steal them, and any credential that has been committed must be rotated. The fix is to load these values from environment variables at runtime instead of a static config object.

```
var sequelize = new Sequelize(process.env.DB_NAME, process.env.DB_USER, process.env.DB_PASS, {
    host: process.env.DB_HOST,
    dialect: process.env.DB_DIALECT
  });
}
```

/tmp/aicr_gh_5qi1ocar/models/index.js:22 (verified)

The .then handler incorrectly receives an error parameter and treats success as an error path, while .catch only logs the failure. This leaves the app running with a broken DB connection, risking runtime errors later. Proper promise handling (or async/await with try/catch) ensures the connection is verified before continuing.

```
sequelize.authenticate()
  .then(() => {
    console.log('Connection has been established successfully.');
  })
  .catch((err) => {
    console.error('Unable to connect to the database:', err);
    // Depending on policy, process.exit(1) or throw to halt startup
    throw err;
  });
```

/tmp/aicr_gh_5qi1ocar/models/index.js:28 (verified)

The sequelize.sync() promise is not fully handled: the .then() callback receives an 'err' parameter that is never checked, and

there is no .catch() to handle rejections. This can lead to unhandled promise rejections, silent failures, and the application continuing in an inconsistent state. The secure-coding guidance emphasizes catching specific errors, logging context, and avoiding patterns that swallow exceptions.

```
sequelize
  .sync( /*{ force: true }*/ ) // Force To re-initialize tables on each run
  .then(function () {
    console.log('It worked!');
  })
  .catch(function (err) {
    console.error('Sequelize sync failed:', err.message);
    // Consider process.exit(1) or graceful shutdown depending on requirements
  });
```

/tmp/aicr_gh_5qi1ocar/models/user.js:22 (verified)

Storing passwords in cleartext (plain STRING column with no hashing) is a critical vulnerability. An attacker who obtains the database can immediately use every user's credentials. The secure approach is to hash passwords with a strong, slow algorithm such as bcrypt before they are saved. Sequelize models should use a beforeCreate / beforeUpdate hook (or the setDataValue pattern) to ensure the password is never persisted in plaintext.

```
        password: {
            type: DataTypes.STRING,
            allowNull: false,
            set(value) {
                const hash = bcrypt.hashSync(value, 12);
                this.setDataValue('password', hash);
            }
        },
```

/tmp/aicr_gh_5qi1ocar/models/user.js:29 (verified)

The role field is defined as a nullable STRING with no enum constraint or validation. This allows any value (including null or arbitrary strings) to be stored, making it impossible for the model to enforce Role-Based Access Control (RBAC). Without an explicit list of permitted roles the application cannot reliably implement least-privilege authorization, violating secure permission assignment practices.

```
        role: {
            type: DataTypes.ENUM('user', 'moderator', 'admin'),
            allowNull: false,
            defaultValue: 'user'
        }
```

/tmp/aicr_gh_5qi1ocar/public/assets/showdown.min.js:1 (verified)

The extension validator (function b) and converter setup rely on throwing Errors for every validation failure and use console.warn for some cases. This provides no centralized, graceful failure path that an application can catch and handle cleanly, especially when user-provided extensions are malformed. Following secure error-handling guidance, we should avoid letting unhandled exceptions bubble up and instead surface a consistent, safe error object that callers can inspect without leaking internal details.

```
d.validateExtension=function(a){var c=b(a,null);if(!c.valid){console.warn(c.error);var e=new Error(c.error);e.isExtensio
```

/tmp/aicr_gh_5qi1ocar/routes/app.js:23 (verified)

The /bulkproducts route passes req.query.legacy directly into the template context. If the legacy view (or the template engine) outputs this value without HTML escaping, an attacker can supply a query string containing script tags or other payloads, resulting in reflected XSS. The fix is to explicitly escape the value before it reaches the template so that even if auto-escaping is disabled the value cannot break out of its HTML context.

```
    router.get('/bulkproducts', authHandler.isAuthenticated, function (req, res) {
        res.render('app/bulkproducts',{legacy: res.locals.escape ? res.locals.escape(req.query.legacy) : String(req.quer
    })
```

/tmp/aicr_gh_5qi1ocar/routes/app.js:39 (flagged by verifier)

The /admin route passes a client-controlled boolean derived from req.user.role directly to the template. Any client-side JavaScript that gates admin UI or actions based on this flag can be trivially bypassed, and the server comment indicates that privileged operations may not re-verify the role on the server. This creates a confused-deputy situation where authorization is effectively performed on the client. The fix is to remove the admin flag from the rendered data so the template cannot rely on it; any admin-only behavior must be enforced by server-side route middleware that re-checks the authenticated user's role on every request.

```
        res.render('app/admin', {
            // admin flag removed; server middleware must re-verify role on each privileged action
        })
    })
```

/tmp/aicr_gh_5qi1ocar/routes/main.js:22 (verified)

req.params.vuln is attacker-controlled and inserted directly into the template data object. If the template engine performs unescaped interpolation or the dictionary lookup fails, the value can be rendered as raw HTML/JS, leading to reflected XSS (CWE-79). The fix is to HTML-escape the value before it reaches the template.

```
vuln_reference: escapeHtml(req.params.vuln) + '/reference',
vulnerabilities: vulnDict
```

/tmp/aicr_gh_5qi1ocar/routes/main.js:24 (flagged by verifier)

The code looks up vulnDict[req.params.vuln] and uses the result directly for the page title without first verifying that the key exists. An attacker-supplied or mistyped vulnerability name therefore produces an incorrect title or can trigger unexpected behavior (e.g. undefined values rendered to HTML). Per CWE-20, all external input must be validated against an explicit allow-list before use; here we simply check whether the key is present and reject the request otherwise.

```
if (!vulnDict.hasOwnProperty(req.params.vuln)) {
    return res.status(404).send('404');
}, function (err, html) {
    if (err) {
        console.log(err)
        res.status(404).send('404')
    } else {
```

/tmp/aicr_gh_5qi1ocar/routes/main.js:46 (verified)

The /forgotpw route is missing the authHandler.isAuthenticated middleware that protects other routes. This allows unauthenticated users to trigger the logout-related logic unnecessarily, wasting resources and potentially exposing side effects. Adding the middleware ensures the route only executes for properly authenticated sessions, following the pattern used elsewhere in the router.

```
router.get('/forgotpw', authHandler.isAuthenticated, function (req, res) {
    res.render('forgotpw')
})
```

/tmp/aicr_gh_5qi1ocar/server.js:19 (verified)

express-fileupload is mounted globally with its default options, which allow any file type and size to be saved to a predictable location on disk. This matches CWE-20 (Improper Input Validation) and creates an arbitrary-file-upload vector that can lead to remote code execution or storage of malicious content. The secure-coding guidance requires explicit allow-list validation of file type, size, destination path, and content before any write occurs.

```
app.use(fileUpload({
  limits: { fileSize: 5 * 1024 * 1024 }, // 5 MiB
  safeFileNames: true,
  preserveExtension: true,
  abortOnLimit: true
}));

// later, inside the route handler:
// if (!req.files || !req.files.upload) return res.status(400).send('No file');
// const file = req.files.upload;
// if (!['image/jpeg','image/png','application/pdf'].includes(file.mimetype)) return res.status(400).send('Bad type');
// const safePath = path.join(uploadDir, file.name.replace(/[^a-zA-Z0-9._-]/g,''));
```

/tmp/aicr_gh_5qi1ocar/server.js:23 (verified)

The default session cookie name ("connect.sid") reveals that the app uses Express with express-session. Attackers can fingerprint the server and launch targeted attacks, similar to an X-Powered-By header. Setting an explicit, non-default name removes this information leak. The provided secure-coding guidance on fingerprinting-style issues (e.g. hard-coded defaults) supports changing the identifier.

```
app.use(session({
  name: 'sessionId',
  secret: 'your-secret',
  resave: false,
  saveUninitialized: false
}));
```

/tmp/aicr_gh_5qi1ocar/server.js:23 (verified)

The express-session middleware is used without an explicit 'domain' option. This can allow cookies to be sent to unintended subdomains or make the session cookie less secure, violating the principle of restricting the cookie's scope to the exact domain where it is needed. The rule flags the default configuration because it does not compare the cookie domain against the requested server domain before setting path attributes.

```
const session = require('express-session');
// ...
app.use(session({
  secret: 'your-secret',
  resave: false,
  saveUninitialized: false,
  cookie: {
    domain: '.example.com', // or 'example.com' as appropriate
    httpOnly: true,
    secure: true
  }
}));
```

/tmp/aicr_gh_5qi1ocar/server.js:23 (verified)

The Express session middleware is using default cookie settings that do not explicitly set an expiration date. Without `expires` (or `maxAge`), the session cookie becomes a persistent cookie that can remain in the browser indefinitely, increasing the window for session hijacking or replay attacks. Setting an explicit short expiration forces re-authentication and

aligns with secure session management practices.

```
app.use(session({
  secret: 'your-secret',
  cookie: {
    expires: new Date(Date.now() + 60 * 60 * 1000) // 1 hour
  }
}));
```

/tmp/aicr_gh_5qi1ocar/server.js:23 (verified)

The Express session middleware is using default cookie settings that do not explicitly enable the httpOnly flag. Without httpOnly, the session cookie can be read by client-side JavaScript, which increases the impact of any XSS vulnerability (CWE-79) by letting an attacker steal the session. Setting httpOnly: true tells the browser to withhold the cookie from JavaScript, providing a strong mitigation for XSS while leaving the cookie available for normal HTTP/S requests.

```
app.use(session({
  secret: 'your-secret',
  cookie: { httpOnly: true }
}));
```

/tmp/aicr_gh_5qi1ocar/server.js:23 (verified)

The express-cookie-session middleware is used with default settings that do not explicitly set the 'path' option. This can cause the session cookie to be sent on every request (including unrelated paths), increasing the risk of session fixation or leakage. Setting an explicit path (commonly '/') ensures the cookie is only attached to the intended routes.

```
app.use(session({
  secret: 'your-secret',
  resave: false,
  saveUninitialized: true,
  cookie: {
    path: '/',
    httpOnly: true,
    secure: true
  }
}));
```

/tmp/aicr_gh_5qi1ocar/server.js:23 (verified)

The express session middleware is used without explicitly setting the 'secure' option on the cookie. Without secure: true the session cookie can be sent over plain HTTP, allowing an attacker to intercept it and hijack the user's session. The rule flags the default insecure configuration; we must tell the browser to transmit the cookie only on HTTPS connections.

```
const session = require('express-session');
app.use(session({
  secret: 'your-secret',
  resave: false,
  saveUninitialized: false,
  cookie: { secure: true }
}));
```

/tmp/aicr_gh_5qi1ocar/server.js:24 (verified)

The express-session middleware is being initialized with a hard-coded secret string directly in the source code. This violates CWE-798 and CWE-259 because the secret could be leaked if the repository is exposed, allowing attackers to forge session cookies or hijack sessions. Secrets should be loaded from environment variables at runtime so they are never committed to version control and can be rotated easily.

```
const session = require('express-session');

app.use(session({
  secret: process.env.SESSION_SECRET || 'fallback-for-dev-only',
  resave: false,
  saveUninitialized: false
}));
```

/tmp/aicr_gh_5qi1ocar/server.js:29 (verified)

The session secret is hard-coded as the weak value 'keyboard cat'. This violates CWE-798 because anyone with access to the source code can decrypt or forge session cookies. Secrets must be loaded from environment variables at runtime so they never appear in version control and can be rotated easily.

```
const sessionSecret = process.env.SESSION_SECRET;
if (!sessionSecret) {
  console.error('SESSION_SECRET environment variable is required');
  process.exit(1);
}

app.use(session({
  secret: sessionSecret,
  resave: false,
  saveUninitialized: false,
  cookie: { secure: process.env.NODE_ENV === 'production' }
}))
```

/tmp/aicr_gh_5qi1ocar/server.js:31 (verified)

The session middleware (via passport.session()) creates and manages session cookies. When cookie.secure is not

explicitly set to true (or the shown configuration leaves it false), the cookie can be sent over plain HTTP. This exposes the session ID to network interception and enables session fixation attacks. Setting secure:true, plus httpOnly and sameSite, tells the browser to transmit the cookie only over HTTPS and adds extra protections.

```
app.use(session({
  secret: 'your-secret',
  resave: false,
  saveUninitialized: false,
  cookie: {
    secure: true,
    httpOnly: true,
    sameSite: 'strict'
  }
}));

// Initialize Passport
app.use(passport.initialize())
app.use(passport.session())
```